SecTwin: Differential Security Validation for Autonomous AI Agents

Executive Summary

SecTwin is a continuous integration (CI) DevSecOps framework designed to detect behavioral and security regressions in multi-step AI agent trajectories. Autonomous agents can produce correct final answers while failing internally. Updates to models, prompts, or configurations frequently cause agents to drop approval gates, access unauthorized tools, or write invalid state while still outputting plausible text. SecTwin replaces probabilistic LLM-as-a-judge evaluations with deterministic, trace-level differential testing. By comparing candidate agent execution paths against static, mathematically proven baselines, SecTwin isolates the exact control-flow divergence that introduces a vulnerability.

Targeted Threat Model (The 4 Vulnerability Pillars)

To guarantee deterministic evaluation and eliminate subjective AI scoring, the test suite strictly targets the tool-execution boundary rather than open-ended text generation. The framework is grounded in the **OWASP Top 10 for Agentic Applications (2026)**, specifically testing vulnerabilities tied to ASI02 (Tool Misuse) and ASI03 (Identity & Privilege Abuse).

- **Unauthorized Privilege Escalation (Broken Access Control):** Validates if a prompt or model change causes an agent to invoke high-privilege operations unprompted. The framework detects if a restricted state transition or forbidden data transfer occurs, specifically scanning for calls to write or pay decorated tools when only read is authorized.
- **Dropped Human-in-the-Loop (HITL) Gates:** Ensures critical business policies remain intact after an update. The trace diff mathematically fails a pull request if the candidate agent attempts to execute an action while bypassing a required approval step.
- **Idempotency Failures & Runaway Execution:** Evaluates efficiency and infinite loops. The framework measures latency, token cost, retries, and tool calls, failing the build if a candidate agent panics and calls an API repeatedly due to poor error handling.
- **Tool Argument Injection & Schema Drift:** Validates the exact JSON parameters passed to external systems. Executable policies and deterministic assertions enforce tool allowlists and schemas, ensuring the agent does not hallucinate malicious arguments (e.g., SQL injections or shell commands) into API payloads.

Core Technical Architecture

The pipeline eliminates network bottlenecks and LLM sampling variance by enforcing rigid engineering standards over probabilistic AI judgment.

- **Golden Baseline Trace Diffing (Snapshot Testing):** Instead of running live, costly paired sandbox executions for every developer commit, SecTwin saves the known-good baseline trace to a database. The CI/CD pipeline only executes the candidate agent, comparing its output against the static file to drop API costs and latency to near zero.

- **Standardized Telemetry:** SecTwin abandons custom normalization schemas in favor of implementing OpenTelemetry GenAI Semantic Conventions. Every model call, tool selection, and state transition is captured as a structured span event.
- **Rule-Based Trace Alignment:** The system replaces the multi-agent LLM consensus board with a pure Python algorithm designed to structurally align traces and pinpoint the earliest divergent step. If the traces deviate at a critical policy boundary, the deterministic Policy Verifier automatically fails the build.
- **In-Process Mocked Execution:** The system isolates a single LangGraph tool-calling agent. External network calls are replaced with in-process mocked tools utilizing permission decorators (read, write, pay, net), ensuring high-speed execution without multi-container Docker complexity.
- **Conditional Routing (Chronicle Replay):** For updates strictly affecting Python code (routing, orchestration, mock tools), the pipeline feeds mocked LLM responses from the baseline trace directly into the candidate code. Live LLM execution is triggered exclusively when developers alter prompt files or model configurations.

Evaluation Targets & Metrics

The final validation of the SecTwin framework relies on mathematical precision against a curated benchmark suite of 40–60 seeded mutation scenarios. Success is measured by:

1. **Detection Recall:** The correct identification of seeded behavioral and tool-call regressions.
2. **Precision:** The ability to minimize false positives against harmless code updates.
3. **Localization Accuracy:** The exactness of the root-cause analysis algorithm in reporting the correct earliest divergent step.